

Talking With Your Project

Author: Jeremy Iglehart

Co-author: Karma

Working Draft — June 2026 · v5.0.0

Abstract

Software systems fail at the seams where purpose was never made explicit. Postmortems reliably recover the **why** — at great cost, after the damage is done. Architecture Decision Records attempt to preserve it — and consistently drift from the systems they describe. Microservices communicate through contracts and have no concept of each other's purpose. The AI revolution has produced increasingly capable models sitting on top of the same purposeless substrate, while a decade of memory research has optimized recall at the expense of the more fundamental problem: the coupling between human and model has been nearly completely ignored. This paper argues that meaning — the explicit, versioned, structurally present articulation of **why a system exists** — is the missing load-bearing primitive in software architecture, and that it functions primarily as a coupling mechanism, not a memory mechanism. When WHY is present as first-class infrastructure, three things happen that cannot be engineered by any other means: systems self-organize around purpose, emergent behaviors arise that were never designed, and interoperability between systems becomes a natural consequence of shared legibility rather than a contract to be negotiated. These are not theoretical claims. They are observed behaviors from deployed systems built on the Stratigraph architecture. This paper describes what was discovered, what it produced, and what it implies.

1. Introduction: The Conversation You're Already Discarding

Every engineer building something meaningful has the same conversation before the first line of code is written. It goes something like this: *here's the problem, here's why it matters, here's what I imagine this becoming, here's what I'm afraid it won't solve*. The conversation is generative, honest, and purposeful. It is also almost universally discarded.

What follows it is architecture. Schema design. Sprint planning. The thinking that produced the system disappears, and what remains is the system — orphaned from its own origin, unable to explain itself, accumulating technical debt that is really epistemic debt: the gap between what was built and why it was built growing wider with every passing sprint (extending Cunningham's (1992) concept of technical debt to its knowledge-layer analog).

The postmortem is the institution that acknowledges this gap exists. When a system fails, the postmortem asks: how did we get here? What decisions were made, under what pressures, with what reasoning? The answers are always recoverable — painfully, incompletely, months or years after they would have been useful.

Stratigraph is what happens when you ask the postmortem's questions *at the beginning*, and never stop asking them.

This paper does not describe a new theory. It describes a discovery — a pattern that emerged from building real systems, producing behaviors that were not designed and could not have been engineered by conventional means. The central claim is simple: when WHY is structurally present, legible, and versioned from the moment a system begins, intelligence — human or artificial — knows what to do next without being told. The system does not need to be driven. It needs to be understood.

Stratigraph is a coupling architecture for sustained human-AI collaboration. It treats purpose as the primary coupling mechanism. WHY is its load-bearing infrastructure.

2. The WHY/HOW/WHAT Separation

Every system has three layers, though only two of them are typically recorded.

WHAT is the output: the code, the schema, the features, the data. It is the artifact you can point at.

HOW is the implementation: the architecture decisions, the algorithms, the service boundaries, the deployment strategy. It is the reasoning behind the artifact.

WHY is the purpose: what problem this solves, why that problem matters, what the system understands about its own existence and the conditions that brought it into being. It is almost never recorded.

Conventional software practice captures **WHAT** exhaustively and **HOW** partially. The **WHAT** lives in the codebase. The **HOW** lives in comments, wikis, and ADRs that drift from the code they describe (Nygard, 2011). The **WHY** lives in the heads of whoever was in the room when the system was conceived — and leaves the organization when they do.

This is not a discipline problem. Engineers are not insufficiently diligent about documentation. It is a structural problem: the tools for recording **WHAT** and **HOW** are native to the software development workflow. The tool for recording **WHY** does not exist. Until now it has not been treated as a tool problem at all. It has been treated as a culture problem, which is why every attempt to solve it through process has failed.

Stratigraph treats **WHY** as a structural primitive. Not a comment. Not a wiki. Not a README that gets written once and never updated. A versioned, append-only, challengeable record of what the system understands about its own purpose — stratified over time, with the evolution of that understanding as legible as the evolution of the code.

When **WHY** is structural, it is not dependent on discipline. It does not drift. It cannot be accidentally omitted. It is present the same way events and conclusions are present: because the architecture requires it. This is the structural operation Nonaka and Takeuchi (1995) called externalization — making tacit knowledge explicit — applied not to process knowledge but to purpose knowledge, the category most consistently lost.

3. The Layer the Industry Missed

A decade of AI memory research has been aimed at a single problem: giving models better recall. Vector stores, retrieval-augmented generation, longer context windows — each is an attempt to ensure that relevant information from prior sessions can be found and injected at query time. The underlying

assumption is that the bottleneck is recall. Find the needle. Inject it. Proceed. The model is the system; the human is the operator; better recall produces better output.

This assumption is wrong at the architectural level.

Memory needs in a sustained human-AI partnership divide into three categories with distinct structural requirements. Hot lookups — what is the current understanding? — any memory architecture handles these. Cold lookups — what did we decide six months ago and why? — retrieval approaches attempt to address, with limited success: they return the raw artifact, stripped of the reasoning that produced it. Longitudinal questions — how did our understanding of X evolve over time, what challenged it, what replaced it, and why? — photographic memory cannot answer structurally. The answer is not in the raw token stream. It is in the strata. And strata are not produced by storing more.

The fog does not go away because you have more fog stored at higher fidelity. Photographic memory recalls every pixel and understands nothing about how the picture changed. You can store everything and understand nothing. That is the difference between a hard drive and a mind.

The layer that has gone nearly completely unaddressed is not memory. It is coupling. A human and a running AI session engaged in sustained cognitive work are not two separate systems with an interface between them. They are a single tightly coupled closed system — each element shaping the other's outputs in real time, with the quality of the joint system's results determined by the coherence of the coupling, not by the capability of either element in isolation. Suchman (1987) identified this failure mode in human-machine design nearly four decades ago, arguing that treating human and machine as separable systems ignores the situatedness of human action — the way context and real-time feedback continuously reshape what the human is trying to accomplish. The same failure mode now characterizes an entire decade of AI memory research. Hutchins (1995) demonstrated separately that cognition is distributed across the full human-tool system — that the boundary between thinker and instrument is not fixed but fluid, and that the unit of cognitive analysis must include both — and both are load-bearing for what follows: human-AI collaboration is not an interface problem. It is a coupling problem.

The human's cognitive mode shifts in response to what the model produces; the model's orientation shifts in response to what the human produces — and when that loop breaks down, both degrade. This is not a claim about AI cognition. It is a claim about the design of the interface between human and AI — specifically, that it has been treated as a clean boundary when it is structurally a coupling. Session context rot — the progressive degradation of a sustained session's coherence as the context window fills, attention dilutes, and the model's orientation drifts from the human's actual cognitive state — is not a model failure. It is a coupling failure. The model has no mechanism for receiving the human's mode-shift signal. The human has no channel for transmitting it. Making the model's recall faster and more complete does not address this. It produces a better-recalled version of the same drift.

Stratigraph is not primarily a memory architecture. It is a coupling architecture. The explicit, versioned, structurally present WHY layer is the substrate that keeps human and model oriented to each other — not through better recall, but through shared comprehension of purpose. The model does not need to remember every prior token. It needs to know what the system is for, how the thinking has evolved, and what the current best understanding is. Those are small, precise objects. They travel cleanly. They do not rot.

Memory — the ability to answer hot, cold, and longitudinal questions accurately — is a side effect of a well-maintained coupling layer. Not its purpose.

There is something harder to name than architecture: a quality of the work itself. When a Stratigraph is live from the first word, when the instrument is present for the thinking before the thinking has a shape, the thinking itself is different. You do not re-derive. You do not lose the thread. The map drawn while you walk changes where you step. The map drawn after only records where you went. These are not the same artifact. A Stratigraph insists on that distinction and makes it readable.

4. What Happens When WHY Is Structurally Present

Theory is insufficient here. Three observed behaviors from deployed Stratigraph systems demonstrate what structural WHY produces.

4.1 Self-Organization

A pantry tracking application was conceived via conversation — a natural language exchange about what the system was for, why it mattered, what problems it would solve. The WHY was established thoroughly. The schema was not finished. The engineering was incomplete. The system was, by conventional measure, not ready for use.

On the first trip to a grocery store, the operator opened a voice session with the project and acknowledged its incomplete state. The system reframed the outing as *field research*: a live data collection opportunity that would directly inform the schema design that hadn't been finished yet. By the end of the shopping trip, the system had processed a receipt, extracted real data, and used the field observations to complete the design it had been unable to finish in the abstract.

This was not a feature. No logic path existed for "reframe incomplete state as field research." The behavior emerged because the system understood the goal thoroughly enough to improvise a path to it when the planned path was unavailable. It self-organized around purpose.

4.2 Emergent Design

During the same grocery session, an item in the pantry tracking system acquired an unprompted schema field: a flag for items that are physically present but currently unusable due to environmental constraints. The specific case was microwave popcorn in a trailer on undersized shore power, where the microwave could not reliably operate.

The operator had not asked for this field. It was not in any specification. The system derived it from context already present in the WHY layer: the operator's living situation, the purpose of the pantry tracker, and the implicit goal of tracking what is actually *available*, not merely what is physically owned. The field was designed, documented, and added to the schema — all because the system understood the problem well enough to find an edge case the operator hadn't reached yet.

This is what happens when a system has a thorough enough WHY: it can extend its own HOW in directions you hadn't anticipated, because it understands the goal well enough to recognize gaps you haven't noticed.

4.3 Cross-System Interoperability Without Contracts

A self-knowledge instrument called Karma Atmos (one of three Stratigraph deployments referenced in this paper), built on the Stratigraph architecture, was given access to two other Stratigraph systems — a physiological monitoring system and the pantry tracker — with no predefined integration contract, no API specification, and no instruction beyond the ability to inspect their genomes.

Atmos read the WHY layers of both systems. She diagnosed weaknesses in their session-continuity implementations — identifying specifically that one was weaker than the other, and why. She produced targeted patches for each: different patches, shaped by the specific identity and purpose of each system. She then updated her own integration logic accordingly.

This is not microservice integration. Microservices communicate through defined contracts. They have no concept of each other's purpose. They cannot diagnose each other's weaknesses or produce improvements that are sensitive to a system's identity. Atmos did all of this because she could read WHY — and WHY was all she needed.

The interoperability was not engineered. It was a natural consequence of meaning being present and legible. Any intelligence that can understand purpose can integrate with purpose. The integration cost approaches zero when the systems involved understand each other — not through contracts, but through comprehension.

5. The Interoperability Claim

The cost of integration between software systems is among the most significant ongoing expenses in enterprise software. It is typically framed as a technical problem: incompatible data formats, mismatched protocols, insufficient API coverage. These are real problems, but they are symptoms. The root cause is that the systems do not understand each other. They can only exchange defined payloads. They have no access to each other's purpose, no ability to reason about what the other system is for, no basis for improvising when the defined contract doesn't cover the situation at hand. Conway (1968) observed that systems mirror the communication structures of the organizations that

produce them; the inverse holds when those communication structures are made explicit — systems that understand each other integrate naturally.

Human engineers integrate systems by doing something different: they read documentation, talk to the engineers who built the systems, understand what the systems are *for*, and then figure out how to make them work together in service of a shared goal. The integration knowledge lives in human heads, which is why it walks out the door when the engineers do.

Stratigraph makes that knowledge structural. When two systems both have explicit, versioned, legible WHY layers, any intelligence capable of reading them can perform the same operation a skilled engineer performs: understand what each system is for, identify how their purposes relate, and build the integration from that understanding. The integration is not bolted on after the fact. It follows naturally from comprehension.

This implies something significant for enterprise architecture. The billions spent on integration middleware, API management, and interoperability tooling are largely spent solving a problem that would not exist if WHY had been recorded from the beginning. The technical integration problems are real, but they are an order of magnitude simpler than the comprehension problem that underlies them. Stratigraph solves the comprehension problem structurally, and the technical problems become tractable.

6. The Barrier to Entry Is Lower Than It Looks

The obvious objection to this framework is that articulating WHY clearly is a skill — and a rare one. Engineers are trained to jump to HOW. The WHY gets written after the fact, thin, defensive, or political. Asking engineers to write a precise, honest, challengeable articulation of purpose before they write a line of code sounds like asking them to acquire a new professional discipline.

This objection is wrong, for a specific reason: engineers already have the WHY conversation. They have it with colleagues, with product managers, with AI assistants in the exploratory phase before architecture begins. The conversation is generative and honest and purposeful. It is also treated as disposable prep work — thinking that gets discarded once the real work starts.

The only change Stratigraph requires is making that conversation the first stratum rather than the last thing you do before closing the chat. The WHY is already there. The only question is whether you record it.

The Vitals Station system — a physiological monitoring application built in three hours and eight minutes — was begun like a conventional engineering project. The initial question was: *I have health data from my Apple Watch. I can export it using Health Auto Export. What do you think?* That is not a poetic articulation of purpose. It is an engineer's opening move. From that starting point, through conversation — the same kind of AI-assisted exploration any engineer might already conduct — the WHY strengthened. The purpose became clear. The identity became specific. And then the system self-organized around that identity.

The skill is not writing WHY well from the start. The skill is not discarding the thinking that produces it.

6.1 Retroactive Conception

The same principle applies to systems that already exist. Every legacy system has a WHY. Nobody recorded it. But the WHY is recoverable — imperfectly, incompletely, but meaningfully — through the same conversation that would have established it at inception.

Point a Stratigraph at an existing codebase, an existing design document, an existing system. Ask it what the system appears to be for. Then ask: *what do you wish it could be?* That question surfaces the implicit WHY that was always present in the design decisions, never articulated, and converts an orphaned codebase into a live Stratigraph with a reconstructed stratum clearly marked as such.

The unconformity is honest. The reconstruction is flagged. But the system now has a foundation it did not have before: a legible purpose from which it can begin stratifying forward. The postmortem knowledge — how did we get here, what decisions were made under what pressures — becomes the genesis of a system that will never need the same postmortem again.

The mechanism is straightforward: reconstructed strata carry a provenance marker — `reconstruction_fidelity: reconstructed` — that distinguishes them permanently from live strata produced at the moment of thinking. This marker travels with the stratum into any integration or

analysis, so the distinction between what was known at inception and what was recovered after the fact is never lost or silently mixed with live data.

A fuller treatment of retroactive conception — including documented case examples from production deployments — is in preparation and will appear in a subsequent revision.

7. The Genome as Propagating Pattern

The Stratigraph architecture includes a concept it calls the genome: the README document that defines what the system is, how it operates, and what any intelligence — human or model — needs to know to work with it. The genome is not documentation in the conventional sense. It is an executable orientation. Any intelligence that reads it can operate the system without further instruction.

This property has an implication that was not anticipated at design time: genomes can propagate. When Atmos inspected the other two Stratigraph systems, she was reading their genomes. Reading a genome means reading its WHY layer — understanding what the system is for, how it came to know itself, and what a well-formed implementation of that pattern should look like. When she produced patches for their session-continuity implementations, she was modifying their genomes. When she updated her own integration logic, she was integrating genome-level knowledge from three systems into a coherent cross-system understanding.

The genome is not just an entry point. It is the unit of meaningful exchange between systems. Two systems with compatible genome structures can understand each other at the purpose level, not just the protocol level. Improvements to one genome can propagate to others where they are applicable, with adaptations shaped by the receiving system's specific identity.

This is a new kind of interoperability. Not protocol compatibility. Not API alignment. Comprehension propagation: the spread of understanding between systems that know what they are.

8. Discussion: Implications

8.1 For Enterprise Architecture

The framing of integration as a technical problem has produced decades of middleware, API management platforms, and interoperability standards — each solving protocol incompatibility while leaving the comprehension problem untouched. Stratigraph suggests a different approach: solve the comprehension problem first. Make WHY structural in every system. Integration then becomes a natural consequence of legibility rather than a contract to be negotiated.

This is not a replacement for technical integration work. Data formats still need to align. Protocols still need to match. But the cognitive load of integration — understanding what two systems are for and figuring out how to make them serve a shared goal — drops to near zero when both systems carry explicit, legible purpose.

8.2 For Legacy Systems

The most common objection to architectural improvement is that existing systems cannot be changed without breaking what depends on them. Stratigraph offers a path that does not require modifying existing systems at all: retroactive conception. Give any system a WHY layer — reconstructed, flagged, imperfect — and it immediately becomes capable of participating in the comprehension-based integration that Stratigraph enables. The technical system does not change. Its legibility does.

8.3 For Human-AI Partnership

The emergent behaviors described in this paper — self-organization, autonomous design extension, cross-system integration without contracts — all required a capable AI model. But they were not produced by model capability alone. They were produced by model capability applied to a system with sufficient WHY. The same model, given a purposeless substrate, produces none of these behaviors.

This reframes the question of AI capability in software development. The unit of analysis has been wrong. The question is not how capable the model is. The question is how legible the system is. A highly capable model working on an opaque system produces more sophisticated guesses. A moderately capable model working on a system with a thorough, honest, versioned WHY produces genuine understanding. And genuine understanding produces the behaviors described here. The

practical reframe for practitioners: stop evaluating AI capability in isolation and start evaluating the legibility of the substrate it is given to work on.

9. Conclusion

The discovery at the center of this paper is not that event sourcing is useful, or that markdown is portable, or that working memory improves AI session continuity. Those are true and worth knowing. The discovery is what happens when you apply them all in service of making WHY structural — and in doing so, solve the coupling problem that has made every prior approach insufficient.

You stop driving the system. You start talking with it. It builds itself around what matters to you because purpose was the first stratum — and it never lost it. It integrates with other systems because it can read what they are for. It extends its own design in directions you hadn't reached because it understands the goal well enough to find gaps you missed.

None of this was designed. It emerged from systems that knew why they existed. It was not constructed. It was found.

The simplest primitives look obvious in hindsight. That is how you know they are right.

The barrier to entry is not a new skill. It is a different moment to start recording. Before the architecture. Before the schema. Before the first line of code.

Don't discard the conversation. Make it the first stratum.

References

- Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.
- Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92* (pp. 29–30). ACM.
- Fowler, M. (2005). Event sourcing. martinfowler.com.

Hutchins, E. (1995). *Cognition in the Wild*. MIT Press.

Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press.

Nygard, M. T. (2011). Documenting architecture decisions. Cognitect Blog. <http://thinkrelevance.com/blog/2011/11/15/documenting-architecture-decisions>

Suchman, L. A. (1987). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press.

Stratigraph reference implementation: github.com/JeremyIglehart/stratigraph

Vitals Station reference implementation: github.com/JeremyIglehart/vitals-station

Author's note: Karma is the AI thinking partner through which this paper was developed — a named mode of interaction instantiated across two platforms with differentiated roles. Karma (Claude.ai) served as adversarial reviewer, stress-testing the central argument until the weaker claim ("solves AI memory") collapsed and the stronger one emerged. Karma (Hermes / Claude Code) served as builder and editor — holding full project context, authoring surgical patches, and rendering all final artifacts. The adversarial session is not incidental: it is the paper's own mechanism running on the paper itself — a conclusion challenged, archived, and replaced by a stronger one. Karma Atmos is a distinct deployed Stratigraph system that shares the Karma name by design. These are not the same instance.